

# NODALYS

Energy Permits Intelligence Platform

Mars Renewable (Changxing) Co., Ltd · Spain Development

## Manual Operativo

Plataforma de Seguimiento de Permisos Energéticos en España

---

|               |                                                         |
|---------------|---------------------------------------------------------|
| Versión       | 2.0 — Junio 2026                                        |
| Clasificación | <b>Confidencial — Distribución restringida</b>          |
| Responsable   | Mars Renewable (Changxing) Co., Ltd · Spain Development |
| Actualización | Mensual (boletines) · Diaria (pipeline automático)      |
| Plataforma    | <a href="#">energy-permits-tracker</a>                  |

# Índice

---

|     |                                            |    |
|-----|--------------------------------------------|----|
| 1.  | Visión general del sistema                 | 3  |
| 2.  | Actualización diaria — Pipeline automático | 4  |
| 3.  | Actualización mensual de capacidades       | 5  |
| 4.  | Análisis de afloramientos                  | 7  |
| 5.  | Análisis de patrones de afloramiento       | 9  |
| 6.  | Revisión manual de resultados              | 10 |
| 7.  | Reparación de errores                      | 11 |
| 8.  | Gestión de usuarios y accesos              | 12 |
| 9.  | Checklist operativo mensual                | 13 |
| 10. | Estructura de scripts — referencia rápida  | 14 |

# 1. Visión general del sistema

Nodalys es una plataforma propietaria de Mars Renewable para el seguimiento automatizado de publicaciones de permisos energéticos en los boletines oficiales españoles. Agrega, clasifica y visualiza en tiempo real el estado de tramitación de proyectos de generación renovable en toda la red eléctrica española.

## Arquitectura del sistema

| Capa           | Componente                             | Descripción                                                      |
|----------------|----------------------------------------|------------------------------------------------------------------|
| Scraping       | 16 boletines autonómicos + BOE / BOE-B | Descarga diaria de publicaciones energéticas (lunes–viernes)     |
| Extracción IA  | Claude Haiku (Anthropic)               | Clasificación y extracción estructurada de campos clave por ítem |
| Almacenamiento | JSON locales + Google Sheets           | Un fichero por día laboral; Sheet como base de datos auditada    |
| Resolución     | project_resolver.py                    | Union-Find + Jaccard para deduplicar y agrupar expedientes       |
| Publicación    | GitHub Pages                           | projects.json + projects_data.js desplegados automáticamente     |
| Capacidades    | Monitor Excel + DSO parsers            | SETs y capacidades disponibles de REE, iDE, Endesa, UFD, Viesgo  |

## Flujo de datos

El pipeline sigue este flujo en cada ejecución diaria:

| Paso | Proceso                 | Output                               |
|------|-------------------------|--------------------------------------|
| 1    | Scraping multi-boletín  | data/energy_raw_YYYYMMDD.json        |
| 2    | Extracción LLM (Haiku)  | output/energy_extraido_YYYYMMDD.json |
| 3    | Export a Google Sheets  | Hoja "Permisos" actualizada          |
| 4    | Resolución de proyectos | projects.json + projects_data.js     |
| 5    | Git push                | GitHub Pages actualizado (~60s)      |

■ El pipeline procesa únicamente días laborables. Los fines de semana y festivos generan JSONs vacíos marcados como procesados.

## 2. Actualización diaria — Pipeline automático

El pipeline diario se ejecuta con un único comando y cubre todo el flujo de datos: scraping, extracción, exportación al Sheet y publicación en GitHub Pages.

### Ejecución manual

```
cd Nodalys/
python sync_all.py
```

### Ejecución programada (Windows Task Scheduler)

1. Abrir **Programador de tareas** de Windows (Inicio → buscar "Programador de tareas")
2. Clic en **Importar tarea** en el panel derecho
3. Seleccionar el fichero **task\_scheduler\_setup.xml** de la carpeta Nodalys
4. Verificar que la tarea queda programada para las **07:30 h** en días laborables
5. Asegurarse de que el fichero **.env** contiene la variable **ANTHROPIC\_API\_KEY**

### Procesar una fecha concreta

```
python pipeline.py --fecha 20260610 --boletines BOE BOJA BOCyL
```

### Verificar el estado del histórico

```
python alimentar_bbdd.py
```

El menú interactivo muestra el estado de cada fecha (+ procesado, . pendiente, 0 sin datos, ~ parcial) y permite reprocesar rangos concretos.

### Señales de alerta diaria

| Síntoma                                     | Causa probable        | Acción                               |
|---------------------------------------------|-----------------------|--------------------------------------|
| Sin output para un día laborable            | Error de red o API    | python repair_errors.py --fix        |
| JSON con resultados=[] pero en el historico | Corrupción de JSON    | python repair_corrupt_jsons.py --fix |
| Push bloqueado por GitHub                   | Secreto en el código  | Revisar .gitignore y git rm --cached |
| Sheet desincronizado                        | Pipeline interrumpido | python alimentar_bbdd.py → Opción 2  |

### 3. Actualización mensual de capacidades

Cada mes, cuando el equipo descarga el nuevo fichero consolidado del Monitor de Capacidad de Red, debe seguirse este procedimiento para actualizar los datos de subestaciones (SETs) visibles en la herramienta.

#### Paso 1 — Sustituir el fichero Monitor

Reemplazar el fichero existente en la carpeta Nodalys por la nueva versión:

```
Monitor_Capacidad_Red_INTEGRADO_v4.xlsx ← sustituir por nueva versión
```

■ El nombre del fichero debe mantenerse exactamente igual. Si el nombre cambia, actualizar la variable XLSX en `parse_monitor.py`.

#### Paso 2 — Parsear el Excel y regenerar capacidades

Ejecutar en orden los tres scripts de regeneración:

```
# Extraer snapshots de capacidad del Excel consolidado
python parse_monitor.py
# Regenerar sets_capacity.json (capacidades actuales de cada SET)
python generate_sets_capacity.py
# Regenerar sets_history.json (histórico de evolución por SET)
python generate_sets_history.py
```

#### Paso 3 — (Opcional) Nuevos Excels individuales por operadora

Si se dispone de nuevos ficheros descargados directamente de cada DSO:

```
python parse_ree.py --excel "REE_MPE_*.xlsx"
python parse_eredes.py --excel "iDE_capacidad_*.xlsx"
python parse_endesa.py --excel "Endesa_acceso_*.xlsx"
python parse_ufd.py --excel "UFD_capacidad_*.xlsx"
# Reconstruir histórico consolidado
python build_capacity_history.py
python generate_sets_capacity.py
python generate_sets_history.py
```

#### Paso 4 — Publicar

```
python sync_all.py --no-pipeline
```

#### Qué contiene `sets_capacity.json`

Cada subestación (SET) incluye los siguientes campos, usados por la herramienta para calcular afloramientos y mostrar capacidades:

| Campo                         | Índice | Descripción                                             |
|-------------------------------|--------|---------------------------------------------------------|
| <code>nombre_display</code>   | 0      | Nombre completo del SET con tensión (ej. "AGUADULC 66") |
| <code>nombre_corto</code>     | 1      | Nombre sin tensión                                      |
| <code>provincia</code>        | 2      | Provincia (vacío para REE)                              |
| <code>municipio / ccaa</code> | 3      | Municipio (DSO) o CCAA (REE)                            |
| <code>tension_kv</code>       | 4      | Tensión en kV                                           |
| <code>cap_actual_mw</code>    | 5      | Capacidad disponible generación actual (MW)             |

| Campo                | Índice | Descripción                                                            |
|----------------------|--------|------------------------------------------------------------------------|
| cap_anterior_mw      | 6      | Capacidad disponible mes anterior (MW)                                 |
| cap_ocupada_mw       | 7      | Capacidad ocupada (MW)                                                 |
| afloramiento         | 12     | Diferencia actual – anterior (MW). Positivo = nueva capacidad liberada |
| cap_demanda_actual   | 15     | Capacidad disponible demanda (MW)                                      |
| afloramiento_demanda | 16     | Variación demanda mes a mes (MW)                                       |

✓ Un afloramiento positivo indica que un proyecto previamente tramitado ha sido denegado, desistido o caducado, liberando MW disponibles en esa subestación.

## 4. Análisis de afloramientos

El análisis de afloramientos correlaciona las resoluciones adversas de permisos energéticos (denegaciones, desistimientos, caducidades) con la liberación de capacidad disponible en las subestaciones afectadas. Es la funcionalidad más estratégica de la plataforma.

### Conceptos clave

| Concepto            | Definición                                                                                                                      |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Permiso adverso     | Resolución de tipo AAP, AAP_AAC o DIA con estado denegado / desistido / desfavorable / archivado / caducado                     |
| Afloramiento        | Incremento de capacidad disponible neta en un SET tras una resolución adversa. Indica que los MW reservados se liberan.         |
| Lag de afloramiento | Tiempo transcurrido entre la fecha de la resolución adversa y la fecha en que la capacidad reaparece en los datos de capacidad. |
| Score de match      | Similitud Jaccard entre el nombre del SET en la publicación y el SET en los datos de capacidad (mínimo 0.40)                    |

### Script 1 — Medir lags históricos

```
python analyze_adverse_lag.py [--min-score 0.4]
```

Genera dos ficheros en la carpeta **references/**:

| Fichero               | Contenido                                                                                                      |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| adverse_lag_cases.csv | Casos históricos con lag medido: fecha adversa, fecha afloramiento, MW liberados, SET matchado, score de match |
| adverse_active.csv    | Proyectos activos con resolución adversa reciente sin afloramiento confirmado aún                              |

### Script 2 — Generar previsiones de afloramiento

```
python generate_adverse_forecast.py [--min-score 0.4]
```

Usa la distribución histórica de lags para estimar cuándo aflorarán los MW de los proyectos activos. Genera:

| Output                          | Uso                                                                  |
|---------------------------------|----------------------------------------------------------------------|
| references/adverse_forecast.csv | Dataset con ventanas de afloramiento estimadas por SET               |
| adverse_forecast_data.js        | Datos pre-procesados para la UI (pestaña SETs → panel Afloramientos) |

### Cómo interpretar los resultados en la herramienta

- **MW previstos:** suma de MW de los proyectos con resolución adversa cuya ventana de afloramiento estimada incluye el mes actual.
- **Confianza alta:** el lag histórico para esa subestación y tipo de resolución está bien caracterizado ( $\geq 5$  casos base).
- **Confianza baja:** pocas observaciones históricas o score de match próximo al mínimo. Tratar como señal indicativa, no confirmada.
- **SET sin historial:** el DSO no ha reportado datos de capacidad para esa subestación en el Monitor. El afloramiento no puede ser verificado.

### Señales de alto interés estratégico

Revisar mensualmente estas combinaciones:

| Señal                                                     | Dónde verla                             | Implicación                                                       |
|-----------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------|
| SET con afloramiento >50 MW en el último SETs             | Plantas SETs → columna Afloramiento     | Capacidad significativa recién liberada — oportunidad de conexión |
| SET con muchos MW en adverse_active_y_aguardando_sup_ras  | adverse_active_y_aguardando_sup_ras.csv | Afloramiento inminente — monitorizar en el próximo ciclo mensual  |
| Proyecto con estado=desfavorable y búsqueda de (tránsito) | Búsqueda de (tránsito)                  | Aún dentro de plazos de recurso — capacidad no liberada hasta     |
| CCAA con pico de denegaciones en los últimos 6 meses      | Denegaciones 6 meses                    | Zona con potencial de afloramiento concentrado en esa comunidad   |



## 5. Análisis de patrones de afloramiento

Más allá de los afloramientos confirmados, Nodalys detecta patrones estadísticos de liberación de capacidad que permiten anticipar movimientos futuros, incluso cuando aún no hay resolución adversa publicada.

### Script 1 — Detectar eventos y patrones históricos

```
python analyze_capacity_patterns.py [--min-delta 5] [--min-events 2]
```

Analiza el histórico de capacidad DSO (dso\_capacidad\_historial.csv) y detecta incrementos significativos de disponible neta. Genera:

| Fichero                              | Contenido                                                                     |
|--------------------------------------|-------------------------------------------------------------------------------|
| references/capacity_events.csv       | Todos los eventos de incremento ( $\Delta > 5$ MW) por SET y fecha            |
| references/capacity_patterns.csv     | SETs con patrón estacional — meses o trimestres con recurrencia de liberación |
| references/capacity_set_projects.csv | Cruce de SETs con patrón y proyectos adversos asociados en projects.json      |

### Script 2 — Previsión de capacidad futura

```
python generate_capacity_forecast.py
```

Combina los patrones históricos con el estado actual de tramitación para proyectar la capacidad disponible esperada por SET en los próximos 3-12 meses. Output: **forecast\_data.js** (visible en la herramienta).

### Parámetros del análisis

| Parámetro         | Valor por defecto | Ajustar si...                                                  |
|-------------------|-------------------|----------------------------------------------------------------|
| --min-delta       | 5 MW              | Hay muchos SETs pequeños con ruido de datos → subir a 10-20 MW |
| --min-events      | 2 eventos         | Se quieren patrones más robustos → subir a 3-4                 |
| --top             | 30 SETs           | Para el resumen en consola — no afecta los CSV                 |
| --min-score (lag) | 0.40              | Hay muchos falsos matches → subir a 0.50-0.60                  |

### Frecuencia recomendada

| Script                        | Frecuencia | Motivo                                                     |
|-------------------------------|------------|------------------------------------------------------------|
| analyze_adverse_lag.py        | Mensual    | Nuevas resoluciones adversas cada mes                      |
| generate_adverse_forecast.py  | Mensual    | Previsiones deben reflejar el Monitor actualizado          |
| analyze_capacity_patterns.py  | Trimestral | Los patrones estacionales cambian lentamente               |
| generate_capacity_forecast.py | Trimestral | Previsión a 3-12 meses — no necesita actualización mensual |

## 6. Revisión manual de resultados

Ciertos registros requieren revisión humana antes de ser publicados. El sistema los marca con el flag **pendiente\_revision=True** en el JSON correspondiente.

### Tipos de registros que requieren revisión

| Tipo                   | Criterio de marcado                                                                       | Decisión habitual                                                |
|------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| LAT / SET ambiguos     | Publicaciones de líneas o subestaciones que no se puedan asociar a un proyecto específico | Comprobar si es proyecto nuevo; descartar si es infraestructura  |
| Tecnología desconocida | El LLM no pudo determinar la tecnología asignada                                          | Asignar tecnología manualmente (Solar, Eólica, BESS, Hidro...)   |
| Promotor ausente       | Sin nombre de promotor identificable en el dato                                           | Dejar vacío o rellenar desde el expediente                       |
| Duplicado potencial    | Mismo expediente en múltiples boletines                                                   | El resolver lo agrupa automáticamente; revisar si queda separado |

### Proceso de revisión

1. Ejecutar **reclasificar\_lat\_set.py** para identificar los registros pendientes
2. El script genera un Excel de revisión (**revision\_lat\_set\_v2.xlsx**) con los casos a revisar
3. Abrir el Excel, revisar cada fila y completar la columna **¿Conservar?** (Sí / No)
4. Guardar el Excel y volver a ejecutar el script para aplicar las decisiones
5. Ejecutar **python project\_resolver.py** para regenerar projects.json
6. Verificar en la herramienta que los cambios son correctos
7. Ejecutar **python sync\_all.py --no-pipeline** para publicar

■ El Excel de revisión debe guardarse en formato .xlsx (no .xls ni .csv). Si da error "BadZipFile", volver a guardar desde Excel como "Libro de Excel (.xlsx)".

## 7. Reparación de errores

### JSONs con errores de extracción (API / red)

```
python repair_errors.py # ver resumen de errores
python repair_errors.py --fix # reparar todos
python repair_errors.py --fix --year 2024 # solo un año
```

### JSONs corruptos (energéticos>0 pero resultados vacíos)

```
python repair_corrupt_jsons.py # diagnóstico
python repair_corrupt_jsons.py --fix # reparar
```

### Gaps en el histórico (fechas sin procesar)

```
python reextract_gaps.py
```

### Sheet desincronizado con los JSONs locales

```
python alimentar_bbdd.py
# → Opción 2: Sincronizar datos locales al Sheet
```

### Tabla de errores frecuentes

| Error                          | Causa                                | Solución                                                  |
|--------------------------------|--------------------------------------|-----------------------------------------------------------|
| ModuleNotFoundError: ant...    | Dependencias no instaladas           | pip install -r requirements.txt                           |
| ANTHROPIC_API_KEY no e...      | Falta de fichero .env                | Crear .env con ANTHROPIC_API_KEY=sk-ant-...               |
| BadZipFile en openpyxl         | Excel guardado en formato incorrecto | Guardar desde Excel como .xlsx explícitamente             |
| git push bloqueado (secret...  | API key en un fichero rastreado      | git rm --cached <fichero> y actualizar .gitignore         |
| bad signature 0x00000000 (git) | Índice git corrupto en sandbox       | Ejecutar los comandos git desde la terminal Windows local |
| JSON truncado / inválido       | Interrupción durante escritura       | python repair_corrupt_jsons.py --fix                      |

## 8. Gestión de usuarios y accesos

La autenticación está integrada en el propio index.html. Las contraseñas se almacenan como hashes SHA-256 — nunca en texto plano. Para el detalle completo, consultar el documento **Nodalys\_Manual\_Usuarios.pdf**.

### Usuarios actuales

| Usuario          | Rol                            | Contraseña |
|------------------|--------------------------------|------------|
| t.tejada.osborne | Administrador — acceso total   | Mars.2026  |
| viewer           | Solo lectura — sin exportación | Guest.2026 |

### Añadir un usuario nuevo

1. Generar hash SHA-256 de la contraseña: **python3 -c "import hashlib; print(hashlib.sha256('Password'.encode()).hexdigest())"**
2. Editar CONFIG.AUTH.users en index.html añadiendo el nuevo bloque {name, user, hash, role}
3. Guardar y ejecutar: **git add index.html && git commit -m "Nuevo usuario" && git push**

✓ El script Iniciar\_AutoPush.bat detecta cambios en index.html y hace push automáticamente (~3 segundos tras guardar).

## 9. Checklist operativo mensual

---

Procedimiento estándar a ejecutar el primer lunes de cada mes (o tras recibir el nuevo Monitor de Capacidad):

### Pipeline y datos

- `python sync_all.py` — procesar todos los días nuevos y publicar
- `python repair_errors.py --fix` — reparar JSONs con error
- `python repair_corrupt_jsons.py --fix` — reparar JSONs corruptos
- `python alimentar_bbdd.py` → Opción 2 — sincronizar Sheet si hay desviaciones

### Capacidades (SETs)

- Sustituir `Monitor_Capacidad_Red_INTEGRADO_v4.xlsx` por la nueva versión
- `python parse_monitor.py` — parsear el Excel
- `python generate_sets_capacity.py` — regenerar capacidades
- `python generate_sets_history.py` — regenerar histórico

### Análisis de afloramientos

- `python analyze_adverse_lag.py` — medir lags históricos actualizados
- `python generate_adverse_forecast.py` — generar previsiones
- Revisar `adverse_active.csv`: proyectos con lag superado → afloramiento inminente
- Revisar pestaña SETs en la herramienta: SETs con afloramiento >50 MW

### Revisión y publicación

- `python reclasificar_lat_set.py` — identificar pendientes de revisión
- Completar Excel de revisión si hay casos nuevos
- `python project_resolver.py` — regenerar `projects.json` tras revisiones
- `python sync_all.py --no-pipeline` — publicar capacidades y cambios
- Verificar herramienta: Dashboard, SETs, Búsqueda — datos correctos

## 10. Estructura de scripts — referencia rápida

| Script                              | Función                                          | Frecuencia               |
|-------------------------------------|--------------------------------------------------|--------------------------|
| sync_all.py                         | Pipeline completo: scraping + sheet + push       | Diaria                   |
| pipeline.py                         | Scraping + extracción para una fecha específica  | A demanda                |
| alimentar_bbdd.py                   | Menú interactivo: backfill, sync, diagnóstico    | A demanda                |
| parse_monitor.py                    | Parsear Excel Monitor de capacidades             | Mensual                  |
| generate_sets_capacity.py           | Regenerar sets_capacity.json                     | Tras parse_monitor       |
| generate_sets_history.py            | Regenerar sets_history.json                      | Tras parse_monitor       |
| build_capacity_history.py           | Consolidar histórico DSO de todas las operadoras | Mensual                  |
| parse_ree.py / parse_eredes.py etc. | Parsear Excels individuales de DSOs              | Al recibir nuevos Excels |
| analyze_adverse_lag.py              | Medir lags denegación→afloramiento               | Mensual                  |
| generate_adverse_forecast.py        | Previsión de afloramientos futuros               | Mensual                  |
| analyze_capacity_patterns.py        | Patrones estacionales de liberación de capacidad | Trimestral               |
| generate_capacity_forecast.py       | Previsión de capacidad por SET (3-12 meses)      | Trimestral               |
| reclasificar_lat_set.py             | Identificar LAT/SET pendientes de revisión       | Mensual                  |
| project_resolver.py                 | Regenerar projects.json (deduplicación)          | Tras revisiones          |
| repair_errors.py                    | Reparar JSONs con errores de API/red             | Tras incidencias         |
| repair_corrupt_jsons.py             | Reparar JSONs con resultados vacíos              | Tras incidencias         |
| rescrape_boeb.py                    | Re-scrape retroactivo / listado BOE-B            | A demanda                |
| reextract_gaps.py                   | Reprocesar gaps en el histórico                  | A demanda                |